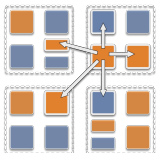


Charm4Py Control Flow

- Despite asynchronous model, we still need control flow
 - for example, synchronizations might be needed for correctness
- One option: enforce dependencies through entry method definitions
- Example: Let's say we have two chares, A and B
 - We want A to always print before B



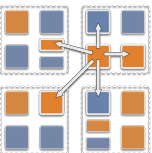
Entry method dependencies

```
class A(Chare):  
  
    def __init__(self, b_proxy):  
        self.b_proxy = b_proxy  
  
    def work(self):  
        print("Hello from A")  
        b_proxy.work()  
  
class B(Chare):  
  
    def work(self):  
        print("Hello from B")  
        exit()
```

```
def main(args):  
    a = Chare(A)  
    b = Chare(B)  
    a.work()
```

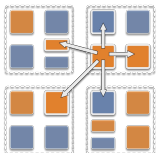
Drawback to entry method linking

- Imagine a large program with many chore types and entry method definitions
 - Need to define dependencies in multiple places across multiple files
 - Hard to maintain
- Helpful to maintain control flow in one place



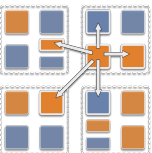
Charm4Py Control Flow

- Best method for managing control flow is through futures
 - Objects that represent values that are unknown at time of creation
- Typical flow
 - Create a future
 - Create a chore, send future to that chore
 - Call an entry method
 - Wait until future is ready



Futures API

- `Future(num_vals=1)`: Creates future
 - `num_vals`: Number of values to wait for
- `get(self)`: Get value of future when ready
 - Blocking call
- `send(self, value=None)`: Sends a value to an object



Futures example

```
class A(Chare):

def __init__(self, b_proxy):
    self.b_proxy = b_proxy

def work(self, future):
    print("Hello from A")
    future.send()

class B(Chare):

    def work(self, future):
        print("Hello from B")
        future.send()
```

```
def main(args):
    a = Chare(A)
    b = Chare(B)
    f = Future()
    a.work(f)
    f.get()
    b.work(f)
    f.get()
    exit()
```

Todo example: Fibonacci

- Compute the Fibonacci sequence recursively
- Given $F_n = F_{n-1} + F_{n-2}$, create chares to compute $n-1$ and $n-2$ terms
- Use futures to wait for inputs, then compute F_n
- Not the most efficient algorithm, but demonstrates use of futures
- Template provided, need to fill in details

